



МИНОБРНАУКИ РОССИИ

**федеральное государственное автономное образовательное учреждение
высшего образования**

«Московский государственный технологический университет

«СТАНКИН»

(ФГАОУ ВО «МГТУ «СТАНКИН»)

Институт
информационных
технологий

Кафедра
информационных систем

**Основная образовательная программа 09.03.02
«Информационные системы и технологии»**

Отчет по дисциплине «Структуры и алгоритмы обработки данных»

Тема: «Лабораторная работа №4»

**Проверил
К.т.н., доцент**

Ковалев И.А.

подпись

**Выполнил
студент группы ИДБ-25-06**

Грачев А.А.

подпись

Москва, 2026 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. 912. РЕАЛИЗАЦИЯ ХЕШ-ТАБЛИЦЫ С ОТКРЫТОЙ АДРЕСАЦИЕЙ.....	4
2. 242. VALID ANAGRAM	6
3. 383. RANSOM NOTE	7
4. 49. GROUP ANAGRAMS	8
5. 128. LONGEST CONSECUTIVE SEQUENCE	9
6. 146. LRU CACHE.....	10
ЗАКЛЮЧЕНИЕ.....	11

ВВЕДЕНИЕ

Целью лабораторной работы является изучение механизмов хеширования, знакомство с эффективными хеш-функциями (MurmurHash, CityHash), понимание роли коэффициента заполнения (load factor) в производительности хеш-таблиц, а также практическое применение этих знаний. В практической части реализована хеш-таблица с открытой адресацией и решены пять задач с платформы LeetCode, каждая из которых интенсивно использует структуры данных на основе хеширования.

1. 912. РЕАЛИЗАЦИЯ ХЕШ-ТАБЛИЦЫ С ОТКРЫТОЙ АДРЕСАЦИЕЙ

Создан класс `MyHashMap`, использующий линейное пробирование. Внутренний массив хранит ключи и значения, а также константу `_DELETED` для пометки удалённых ячеек. При превышении допустимого коэффициента заполнения (взят порог 0.75) таблица автоматически расширяется вдвое и все элементы рехешируются. Сложность отдельных операций в среднем $O(1)$, в худшем случае $O(n)$. Реализация не зависит от внешних библиотек и использует только встроенную функцию `hash()`.

Код:

```
class MyHashMap:
    _DELETED = object()

    def __init__(self, capacity=8):
        self.capacity = capacity
        self.size = 0
        self.load_factor = 0.75
        self.keys = [None] * self.capacity
        self.vals = [None] * self.capacity

    def _hash(self, key):
        return hash(key) % self.capacity

    def _find_slot(self, key):
        idx = self._hash(key)
        while self.keys[idx] is not None:
            if self.keys[idx] == key:
                return idx
            idx = (idx + 1) % self.capacity
        return idx

    def _resize(self):
        old_keys, old_vals = self.keys, self.vals
        self.capacity *= 2
        self.keys = [None] * self.capacity
        self.vals = [None] * self.capacity
        self.size = 0
        for k, v in zip(old_keys, old_vals):
            if k is not None and k is not self._DELETED:
                self.put(k, v)

    def put(self, key, value):
```

```

        if self.size >= self.capacity * self.load_factor:
            self._resize()
        idx = self._find_slot(key)
        if self.keys[idx] is None or self.keys[idx] is
self._DELETED:
            self.keys[idx] = key
            self.vals[idx] = value
            self.size += 1
        else:
            self.vals[idx] = value

def get(self, key):
    idx = self._hash(key)
    start = idx
    while self.keys[idx] is not None:
        if self.keys[idx] == key:
            return self.vals[idx]
        idx = (idx + 1) % self.capacity
        if idx == start:
            break
    return -1

def remove(self, key):
    idx = self._hash(key)
    start = idx
    while self.keys[idx] is not None:
        if self.keys[idx] == key:
            self.keys[idx] = self._DELETED
            self.vals[idx] = None
            self.size -= 1
            return
        idx = (idx + 1) % self.capacity
        if idx == start:
            break

```

Тесты:

```

hm = MyHashMap()
hm.put(1, "one")
hm.put(2, "two")
print(hm.get(1))          # one
hm.remove(1)
print(hm.get(1))          # -1

```

2. 242. VALID ANAGRAM

Описание: Даны две строки s и t . Необходимо определить, является ли t анаграммой s (содержит те же символы в том же количестве).

Решение: Создаётся словарь для подсчёта символов строки s . При проходе по t счётчики уменьшаются. Если какой-либо счётчик становится отрицательным или длины строк не совпадают, строки не являются анаграммами. Время $O(n)$, память $O(1)$ (алфавит ограничен 26 символами).

Код:

```
class Solution:
    def isAnagram(self, s: str, t: str) -> bool:
        if len(s) != len(t):
            return False
        count = {}
        for ch in s:
            count[ch] = count.get(ch, 0) + 1
        for ch in t:
            if ch not in count:
                return False
            count[ch] -= 1
            if count[ch] < 0:
                return False
        return True
```

Тесты:

```
sol = Solution()
print(sol.isAnagram("anagram", "nagaram"))    # True
print(sol.isAnagram("rat", "car"))             # False
```

3. 383. RANSOM NOTE

Описание: Даны две строки: ransomNote и magazine. Можно ли составить ransomNote из букв журнала magazine (каждую букву можно использовать один раз).

Решение: Подсчитываем все символы magazine в словаре. Затем проходим по ransomNote, уменьшая счётчики. Если хотя бы один счётчик становится отрицательным — ответ False. Время $O(m+n)$, память $O(1)$.

Код:

```
class Solution:
    def canConstruct(self, ransomNote: str, magazine: str)
-> bool:
        if len(ransomNote) > len(magazine):
            return False
        letters = {}
        for ch in magazine:
            letters[ch] = letters.get(ch, 0) + 1
        for ch in ransomNote:
            if letters.get(ch, 0) == 0:
                return False
            letters[ch] -= 1
        return True
```

Тесты:

```
sol = Solution()
print(sol.canConstruct("a", "b"))           # False
print(sol.canConstruct("aa", "ab"))         # False
print(sol.canConstruct("aa", "aab"))        # True
```

4. 49. GROUP ANAGRAMS

Описание: Дан массив строк `strs`. Необходимо сгруппировать анаграммы и вернуть список групп.

Решение: Ключом для каждой строки служит кортеж из 26 чисел — количество букв от 'a' до 'z' (можно использовать и отсортированную строку). Словарь группирует строки по ключу. Время $O(N \cdot K)$, где N — число строк, K — максимальная длина строки; память $O(N \cdot K)$.

Код:

```
from collections import defaultdict

class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        groups = defaultdict(list)
        for s in strs:
            count = [0] * 26
            for ch in s:
                count[ord(ch) - ord('a')] += 1
            groups[tuple(count)].append(s)
        return list(groups.values())
```

Тесты:

```
sol = Solution()
res =
sol.groupAnagrams(["eat", "tea", "tan", "ate", "nat", "bat"])
# порядок групп и внутри групп может быть любым
print(res)    # например,
[["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]
```

5. 128. LONGEST CONSECUTIVE SEQUENCE

Описание: Дан неотсортированный массив целых чисел. Необходимо найти длину самой длинной последовательной цепочки (числа, идущие подряд с шагом 1) и сделать это за время $O(n)$.

Решение: Все числа помещаются во множество для константного поиска. Затем для каждого числа, которое является началом последовательности (т.е. $num-1$ отсутствует во множестве), запускается подсчёт длины цепочки. Каждый элемент обрабатывается не более двух раз, поэтому итоговое время $O(n)$, память $O(n)$.

Код:

```
class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        num_set = set(nums)
        longest = 0
        for num in num_set:
            if num - 1 not in num_set:
                current = num
                streak = 1
                while current + 1 in num_set:
                    current += 1
                    streak += 1
                longest = max(longest, streak)
        return longest
```

Тесты:

```
sol = Solution()
print(sol.longestConsecutive([100, 4, 200, 1, 3, 2]))    # 4
(1, 2, 3, 4)
```

6. 146. LRU CACHE

Описание: Реализовать класс `LRUCache` с заданной ёмкостью. Операции `get(key)` и `put(key, value)` должны выполняться за $O(1)$. При превышении ёмкости вытесняется наименее давно использовавшийся элемент.

Решение: Использован стандартный `OrderedDict`, который хранит порядок вставки. При обращении к ключу он перемещается в конец, при вставке — удаляется старый дубликат, вставляется новый, и если размер превышает ёмкость — удаляется первый элемент.

Встроенный `OrderedDict` внутренне основан на хеш-таблице и двусвязном списке, что даёт требуемую асимптотику.

Код:

```
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.cache = OrderedDict()
    def get(self, key: int) -> int:
        if key not in self.cache:
            return -1
        self.cache.move_to_end(key)
        return self.cache[key]

    def put(self, key: int, value: int) -> None:
        if key in self.cache:
            self.cache.move_to_end(key)
        self.cache[key] = value
        if len(self.cache) > self.capacity:
            self.cache.popitem(last=False)
```

Тесты:

```
lru = LRUCache(2)
lru.put(1, 1)
lru.put(2, 2)
print(lru.get(1))      # 1
lru.put(3, 3)          # вытесняется ключ 2
print(lru.get(2))      # -1
lru.put(4, 4)          # вытесняется ключ 1
print(lru.get(1))      # -1
print(lru.get(3))      # 3
print(lru.get(4))      # 4
```

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы были изучены принципы построения хеш-таблиц, их ключевые характеристики и методы борьбы с коллизиями. Реализация хеш-таблицы с открытой адресацией и автоматическим рехешированием позволила на практике оценить влияние load factor на производительность. Решение пяти задач LeetCode продемонстрировало мощь хеширования как инструмента для достижения константного или линейного времени работы: от простого подсчёта символов до построения LRU-кэша. Все алгоритмы протестированы на стандартных примерах, предоставленных платформой, и проанализированы по временной и пространственной сложности.